

dsbda-lab-ass-7

April 3, 2026

1 Assignment 7

1.1 Import Required Library

```
[1]: import nltk
nltk.download('stopwords')
nltk.download('punkt')
nltk.download('wordnet')
nltk.download('averaged_perceptron_tagger')
nltk.download('punkt_tab')
nltk.download('averaged_perceptron_tagger_eng')
```

```
[nltk_data] Downloading package stopwords to
[nltk_data] C:\Users\mehat\AppData\Roaming\nltk_data...
[nltk_data] Package stopwords is already up-to-date!
[nltk_data] Downloading package punkt to
[nltk_data] C:\Users\mehat\AppData\Roaming\nltk_data...
[nltk_data] Package punkt is already up-to-date!
[nltk_data] Downloading package wordnet to
[nltk_data] C:\Users\mehat\AppData\Roaming\nltk_data...
[nltk_data] Package wordnet is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data] C:\Users\mehat\AppData\Roaming\nltk_data...
[nltk_data] Package averaged_perceptron_tagger is already up-to-
[nltk_data] date!
[nltk_data] Downloading package punkt_tab to
[nltk_data] C:\Users\mehat\AppData\Roaming\nltk_data...
[nltk_data] Package punkt_tab is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data] C:\Users\mehat\AppData\Roaming\nltk_data...
[nltk_data] Package averaged_perceptron_tagger_eng is already up-to-
[nltk_data] date!
```

```
[1]: True
```

1.2 1. Tokenize text using NLTK in python

```
[2]: para='Rajgad (literal meaning Ruling Fort) is a hill fort situated in the Pune_
↳district of Maharashtra, India. Formerly known as Murumdev, the fort was the_
↳capital of the Maratha Empire under the rule of Chatrapati Shivaji Maharaj_
↳for almost 26 years, after which the capital was moved to the Raigad Fort.
↳[1] Treasures discovered from an adjacent fort called Torna were used to_
↳completely build and fortify the Rajgad Fort.'
```

```
[3]: print(para)
```

Rajgad (literal meaning Ruling Fort) is a hill fort situated in the Pune district of Maharashtra, India. Formerly known as Murumdev, the fort was the capital of the Maratha Empire under the rule of Chatrapati Shivaji Maharaj for almost 26 years, after which the capital was moved to the Raigad Fort.[1] Treasures discovered from an adjacent fort called Torna were used to completely build and fortify the Rajgad Fort.

```
[4]: para.split()
```

```
[4]: ['Rajgad',
      '(literal',
      'meaning',
      'Ruling',
      'Fort)',
      'is',
      'a',
      'hill',
      'fort',
      'situated',
      'in',
      'the',
      'Pune',
      'district',
      'of',
      'Maharashtra,',
      'India.',
      'Formerly',
      'known',
      'as',
      'Murumdev,',
      'the',
      'fort',
      'was',
      'the',
      'capital',
      'of',
```

'the',
'Mara',
'tha',
'Empire',
'under',
'the',
'rule',
'of',
'Chatrapati',
'Shivaji',
'Maharaj',
'for',
'almost',
'26',
'years,',
'after',
'w',
'hich',
'the',
'capital',
'was',
'moved',
'to',
'the',
'Raigad',
'Fort.[1]',
'Treasures',
'discovered',
'from',
'an',
'adjac',
'ent',
'fort',
'called',
'Torna',
'were',
'used',
'to',
'completely',
'build',
'and',
'fortify',
'the',
'Rajgad',
'Fort.']

```
[5]: from nltk.tokenize import sent_tokenize
     from nltk.tokenize import word_tokenize
```

```
[6]: sent=sent_tokenize(para)
```

```
[7]: sent[2]
```

```
[7]: '[1] Treasures discovered from an adjacent fort called Torna were used to
      completely build and fortify the Rajgad Fort.'
```

```
[8]: words=word_tokenize(para)
```

```
[9]: words
```

```
[9]: ['Rajgad',
      '(',
      'literal',
      'meaning',
      'Ruling',
      'Fort',
      ')',
      'is',
      'a',
      'hill',
      'fort',
      'situated',
      'in',
      'the',
      'Pune',
      'district',
      'of',
      'Maharashtra',
      ',',
      'India',
      '.',
      'Formerly',
      'known',
      'as',
      'Murumdev',
      ',',
      'the',
      'fort',
      'was',
      'the',
      'capital',
      'of',
      'the',
```

'Mara',
'tha',
'Empire',
'under',
'the',
'rule',
'of',
'Chatrapati',
'Shivaji',
'Maharaj',
'for',
'almost',
'26',
'years',
,',
'after',
'w',
'hich',
'the',
'capital',
'was',
'moved',
'to',
'the',
'Raigad',
'Fort',
'.',
'[',
'1',
'],'
'Treasures',
'discovered',
'from',
'an',
'adjac',
'ent',
'fort',
'called',
'Torna',
'were',
'used',
'to',
'completely',
'build',
'and',
'fortify',
'the',

```
'Rajgad',  
'Fort',  
'.']
```

1.3 2. Removing stop words with NLTK in Python

```
[10]: from nltk.corpus import stopwords
```

```
[11]: swords= stopwords.words('english')
```

```
[12]: swords
```

```
[12]: ['a',  
      'about',  
      'above',  
      'after',  
      'again',  
      'against',  
      'ain',  
      'all',  
      'am',  
      'an',  
      'and',  
      'any',  
      'are',  
      'aren',  
      "aren't",  
      'as',  
      'at',  
      'be',  
      'because',  
      'been',  
      'before',  
      'being',  
      'below',  
      'between',  
      'both',  
      'but',  
      'by',  
      'can',  
      'couldn',  
      "couldn't",  
      'd',  
      'did',  
      'didn',  
      "didn't",  
      'do',
```

'does',
'doesn',
"doesn't",
'doing',
'don',
"don't",
'down',
'during',
'each',
'few',
'for',
'from',
'further',
'had',
'hadn',
"hadn't",
'has',
'hasn',
"hasn't",
'have',
'haven',
"haven't",
'having',
'he',
"he'd",
"he'll",
'her',
'here',
'hers',
'herself',
"he's",
'him',
'himself',
'his',
'how',
'i',
"i'd",
'if',
"i'll",
"i'm",
'in',
'into',
'is',
'isn',
"isn't",
'it',
"it'd",

"it'll",
"it's",
'its',
'itself',
"i've",
'just',
'll',
'm',
'ma',
'me',
'mightn',
"mightn't",
'more',
'most',
'mustn',
"mustn't",
'my',
'myself',
'needn',
"needn't",
'no',
'nor',
'not',
'now',
'o',
'of',
'off',
'on',
'once',
'only',
'or',
'other',
'our',
'ours',
'ourselves',
'out',
'over',
'own',
're',
's',
'same',
'shan',
"shan't",
'she',
"she'd",
"she'll",
"she's",

'should',
'shouldn',
"shouldn't",
"should've",
'so',
'some',
'such',
't',
'than',
'that',
"that'll",
'the',
'their',
'theirs',
'them',
'themselves',
'then',
'there',
'these',
'they',
"they'd",
"they'll",
"they're",
"they've",
'this',
'those',
'through',
'to',
'too',
'under',
'until',
'up',
've',
'very',
'was',
'wasn',
"wasn't",
'we',
"we'd",
"we'll",
"we're",
'were',
'weren',
"weren't",
"we've",
'what',
'when',

```
'where',  
'which',  
'while',  
'who',  
'whom',  
'why',  
'will',  
'with',  
'won',  
"won't",  
'wouldn',  
"wouldn't",  
'y',  
'you',  
"you'd",  
"you'll",  
'your',  
"you're",  
'yours',  
'yourself',  
'yourselves',  
"you've"]
```

```
[13]: x=[word for word in words if word not in swords]
```

```
[14]: x
```

```
[14]: ['Rajgad',  
      '(',  
      'literal',  
      'meaning',  
      'Ruling',  
      'Fort',  
      ')',  
      'hill',  
      'fort',  
      'situated',  
      'Pune',  
      'district',  
      'Maharashtra',  
      ',',  
      'India',  
      '.',  
      'Formerly',  
      'known',  
      'Murumdev',  
      ',',
```

```

'fort',
'capital',
'Mara',
'tha',
'Empire',
'rule',
'Chatrapati',
'Shivaji',
'Maharaj',
'almost',
'26',
'years',
',',
'w',
'hich',
'capital',
'moved',
'Raigad',
'Fort',
'.',
'[',
'1',
']',
'Treasures',
'discovered',
'adjac',
'ent',
'fort',
'called',
'Torna',
'used',
'completely',
'build',
'fortify',
'Rajgad',
'Fort',
'.']

```

```
[15]: x=[word for word in words if word.lower() not in swords]
```

```
[16]: x
```

```
[16]: ['Rajgad',
'(',
'literal',
'meaning',
'Ruling',

```

'Fort',
)',
'hill',
'fort',
'situated',
'Pune',
'district',
'Maharashtra',
,',
'India',
,',
'Formerly',
'known',
'Murumdev',
,',
'fort',
'capital',
'Mara',
'tha',
'Empire',
'rule',
'Chatrapati',
'Shivaji',
'Maharaj',
'almost',
'26',
'years',
,',
'w',
'hich',
'capital',
'moved',
'Raigad',
'Fort',
,',
'[',
'1',
'],'
'Treasures',
'discovered',
'adjac',
'ent',
'fort',
'called',
'Torna',
'used',
'completely',

```
'build',  
'fortify',  
'Rajgad',  
'Fort',  
'.']
```

1.4 4. Stemming words with NLTK

```
[17]: from nltk.stem import PorterStemmer
```

```
[18]: ps=PorterStemmer()
```

```
[19]: ps.stem('working')
```

```
[19]: 'work'
```

```
[20]: y=[ps.stem(word) for word in x]
```

```
[21]: y
```

```
[21]: ['rajgad',  
      '(',  
      'liter',  
      'mean',  
      'rule',  
      'fort',  
      ')',  
      'hill',  
      'fort',  
      'situat',  
      'pune',  
      'district',  
      'maharashtra',  
      ',',  
      'india',  
      '.',  
      'formerli',  
      'known',  
      'murumdev',  
      ',',  
      'fort',  
      'capit',  
      'mara',  
      'tha',  
      'empir',  
      'rule',  
      'chatrapati',
```

```

'shivaji',
'maharaj',
'almost',
'26',
'year',
',',
'w',
'hich',
'capit',
'move',
'raigad',
'fort',
'.',
'[',
'1',
']',
'treasure',
'discover',
'adjacent',
'entire',
'fort',
'call',
'turn',
'use',
'complete',
'build',
'fortify',
'rajgad',
'fort',
'.']

```

1.5 3. Lemmatization with NLTK

```
[22]: from nltk.stem import WordNetLemmatizer
```

```
[23]: wnl=WordNetLemmatizer()
```

```
[24]: wnl.lemmatize('working',pos='v')
```

```
[24]: 'work'
```

```
[25]: print(ps.stem('went'))
print(wnl.lemmatize('went',pos='v'))
```

```

went
go

```

```
[26]: z=[wnl.lemmatize(word,pos='v') for word in x]
```

```
[27]: z
```

```
[27]: ['Rajgad',  
      '(',  
      'literal',  
      'mean',  
      'Ruling',  
      'Fort',  
      ')',  
      'hill',  
      'fort',  
      'situate',  
      'Pune',  
      'district',  
      'Maharashtra',  
      ',',  
      'India',  
      '.',  
      'Formerly',  
      'know',  
      'Murumdev',  
      ',',  
      'fort',  
      'capital',  
      'Mara',  
      'tha',  
      'Empire',  
      'rule',  
      'Chatrapati',  
      'Shivaji',  
      'Maharaj',  
      'almost',  
      '26',  
      'years',  
      ',',  
      'w',  
      'hich',  
      'capital',  
      'move',  
      'Raigad',  
      'Fort',  
      '.',  
      '[',  
      '1',  
      '']
```

```
'Treasures',  
'discover',  
'adjac',  
'ent',  
'fort',  
'call',  
'Torna',  
'use',  
'completely',  
'build',  
'fortify',  
'Rajgad',  
'Fort',  
'.'
```

```
[28]: import string
```

```
[29]: string.punctuation
```

```
[29]: '!"#$%&\'()*+,-./:;<=>?@[\\]^_`{|}~'
```

```
[30]: t=[word for word in words if word not in string.punctuation]
```

```
[31]: t
```

```
[31]: ['Rajgad',  
      'literal',  
      'meaning',  
      'Ruling',  
      'Fort',  
      'is',  
      'a',  
      'hill',  
      'fort',  
      'situated',  
      'in',  
      'the',  
      'Pune',  
      'district',  
      'of',  
      'Maharashtra',  
      'India',  
      'Formerly',  
      'known',  
      'as',  
      'Murumdev',  
      'the',
```

'fort',
'was',
'the',
'capital',
'of',
'the',
'Mara',
'tha',
'Empire',
'under',
'the',
'rule',
'of',
'Chatrapati',
'Shivaji',
'Maharaj',
'for',
'almost',
'26',
'years',
'after',
'w',
'hich',
'the',
'capital',
'was',
'moved',
'to',
'the',
'Raigad',
'Fort',
'1',
'Treasures',
'discovered',
'from',
'an',
'adjac',
'ent',
'fort',
'called',
'Torna',
'were',
'used',
'to',
'completely',
'build',
'and',

```
'fortify',  
'the',  
'Rajgad',  
'Fort']
```

```
[32]: from nltk import pos_tag
```

```
[33]: pos_tag(t)
```

```
[33]: [('Rajgad', 'NNP'),  
      ('literal', 'JJ'),  
      ('meaning', 'NN'),  
      ('Ruling', 'NNP'),  
      ('Fort', 'NNP'),  
      ('is', 'VBZ'),  
      ('a', 'DT'),  
      ('hill', 'NN'),  
      ('fort', 'NN'),  
      ('situated', 'VBN'),  
      ('in', 'IN'),  
      ('the', 'DT'),  
      ('Pune', 'NNP'),  
      ('district', 'NN'),  
      ('of', 'IN'),  
      ('Maharashtra', 'NNP'),  
      ('India', 'NNP'),  
      ('Formerly', 'RB'),  
      ('known', 'VBN'),  
      ('as', 'IN'),  
      ('Murumdev', 'NNP'),  
      ('the', 'DT'),  
      ('fort', 'NN'),  
      ('was', 'VBD'),  
      ('the', 'DT'),  
      ('capital', 'NN'),  
      ('of', 'IN'),  
      ('the', 'DT'),  
      ('Mara', 'NNP'),  
      ('tha', 'NN'),  
      ('Empire', 'NNP'),  
      ('under', 'IN'),  
      ('the', 'DT'),  
      ('rule', 'NN'),  
      ('of', 'IN'),  
      ('Chatrapati', 'NNP'),  
      ('Shivaji', 'NNP'),  
      ('Maharaj', 'NNP'),
```

```

('for', 'IN'),
('almost', 'RB'),
('26', 'CD'),
('years', 'NNS'),
('after', 'IN'),
('w', 'NN'),
('hich', 'VBD'),
('the', 'DT'),
('capital', 'NN'),
('was', 'VBD'),
('moved', 'VBN'),
('to', 'TO'),
('the', 'DT'),
('Raigad', 'NNP'),
('Fort', 'NNP'),
('1', 'CD'),
('Treasures', 'NNS'),
('discovered', 'VBN'),
('from', 'IN'),
('an', 'DT'),
('adjac', 'JJ'),
('ent', 'NN'),
('fort', 'NN'),
('called', 'VBN'),
('Torna', 'NNP'),
('were', 'VBD'),
('used', 'VBN'),
('to', 'TO'),
('completely', 'RB'),
('build', 'VB'),
('and', 'CC'),
('fortify', 'VB'),
('the', 'DT'),
('Rajgad', 'NNP'),
('Fort', 'NNP')]

```

1.6 5. Calculate Term Frequency - Inverse Document Frequency (TF-IDF) for each word

```
[34]: from sklearn.feature_extraction.text import TfidfVectorizer
```

```
[35]: tfidf = TfidfVectorizer()
```

```
[36]: v=tfidf.fit_transform(t)
```

```
[37]: v.shape
```

[37]: (73, 52)

```
[38]: import pandas as pd
      pd.DataFrame(v)
```

```
[38]:
```

	0
0	(0, 37)\t1.0
1	(0, 27)\t1.0
2	(0, 31)\t1.0
3	(0, 39)\t1.0
4	(0, 18)\t1.0
..	...
68	(0, 5)\t1.0
69	(0, 19)\t1.0
70	(0, 43)\t1.0
71	(0, 37)\t1.0
72	(0, 18)\t1.0

[73 rows x 1 columns]